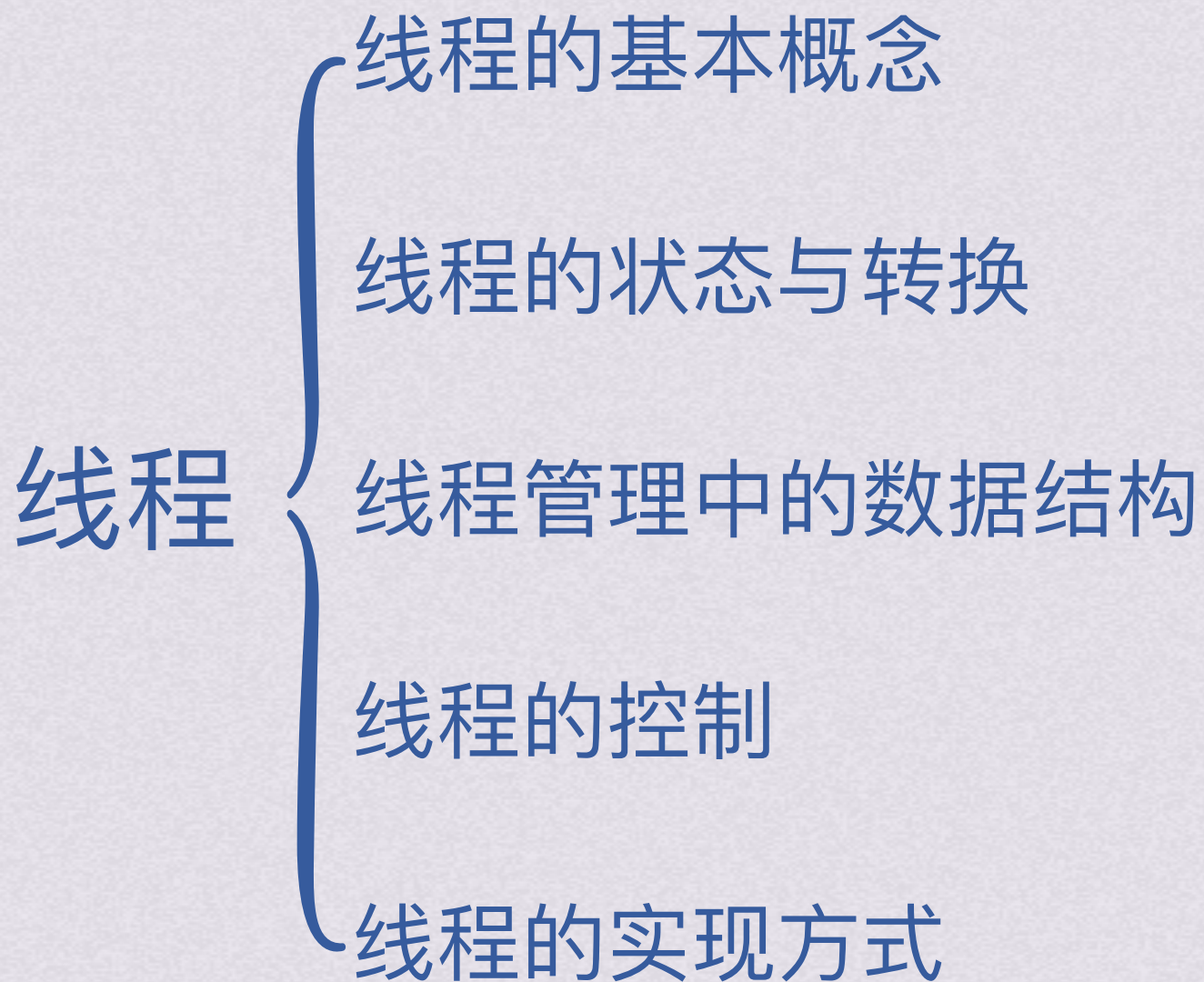


bok25

**基
礎**

課、操作系統





线程的基本概念

- 线程的状态与转换
- 线程管理中的数据结构
- 线程的控制
- 线程的实现方式



线程的基本概念

一个进程只有一个执行点(一个程序计数器, 用来存放要执行的指令), 但多线程(multi-threaded)进程会有多个执行点(多个[逻辑上的]程序计数器, 每个都用于取指令和执行)

每个线程类似于独立的进程, 只有一点区别:它们共享进程的地址空间, 从而能够访问相同的数据



单线程的地址空间



多线程的地址空间



线程的基本概念

一个进程只有一个执行点(一个程序计数器, 用来存放要执行的指令), 但多线程(multi-threaded)进程会有多个执行点(多个[逻辑上的]程序计数器, 每个都用于取指令和执行)

每个线程类似于独立的进程, 只有一点区别:它们共享进程的地址空间, 从而能够访问相同的数据



单线程的地址空间



多线程的地址空间

每个线程都有自己独立的栈, 并且线程之间没有做保护措施, 每个线程的堆栈可以被其他线程读写甚至清除, 由一个线程打开的文件也可以被其他线程读写。

线程只是作为独立调度和分派的单位, 但是它们没有独立的地址空间, 也没有掌握资源, 完全依附于进程



线程的基本概念

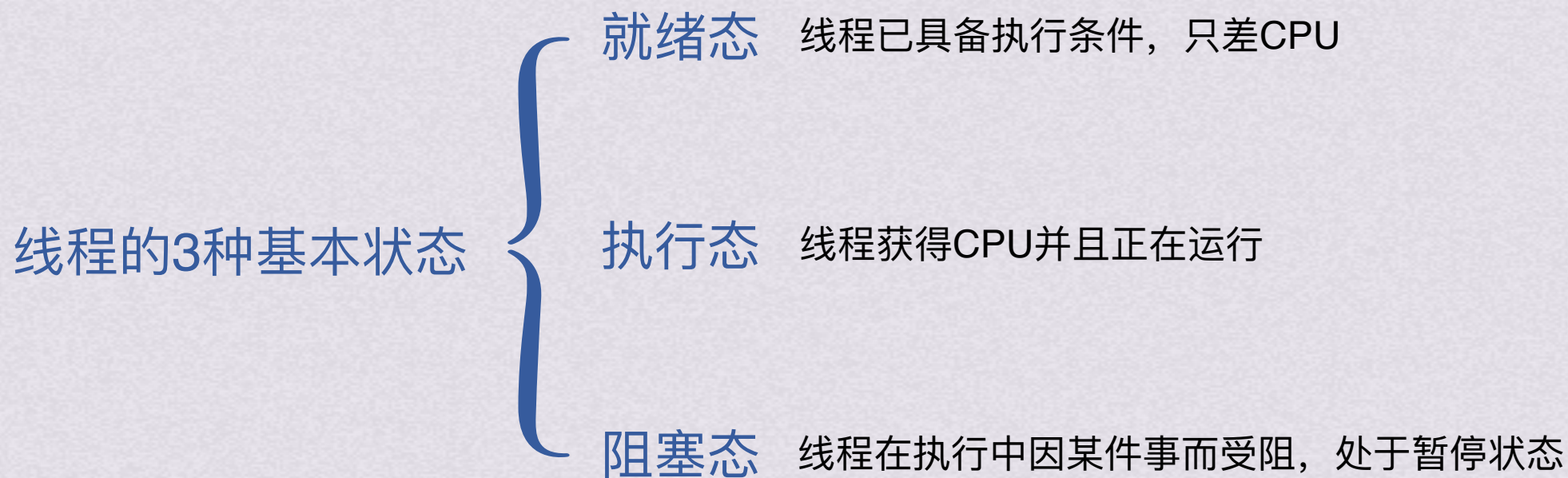
- 线程的状态与转换
- 线程管理中的数据结构
- 线程的控制
- 线程的实现方式



线程的状态与转换



线程的状态与转换





线程的状态与转换

线程的3种基本状态

就绪态

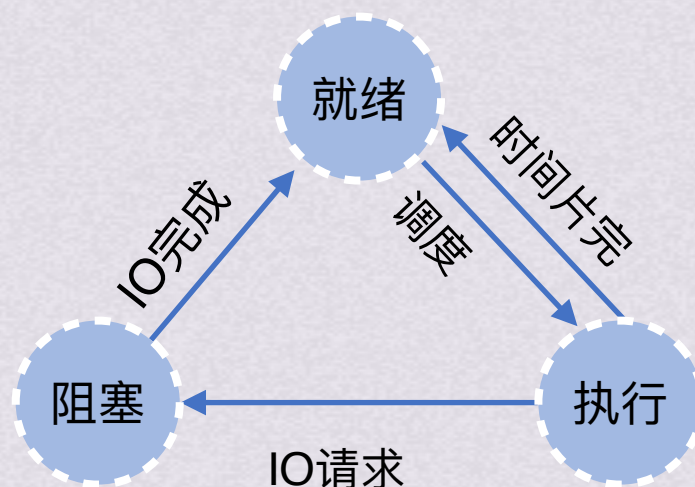
线程已具备执行条件，只差CPU

执行态

线程获得CPU并且正在运行

阻塞态

线程在执行中因某件事而受阻，处于暂停状态





线程的状态与转换

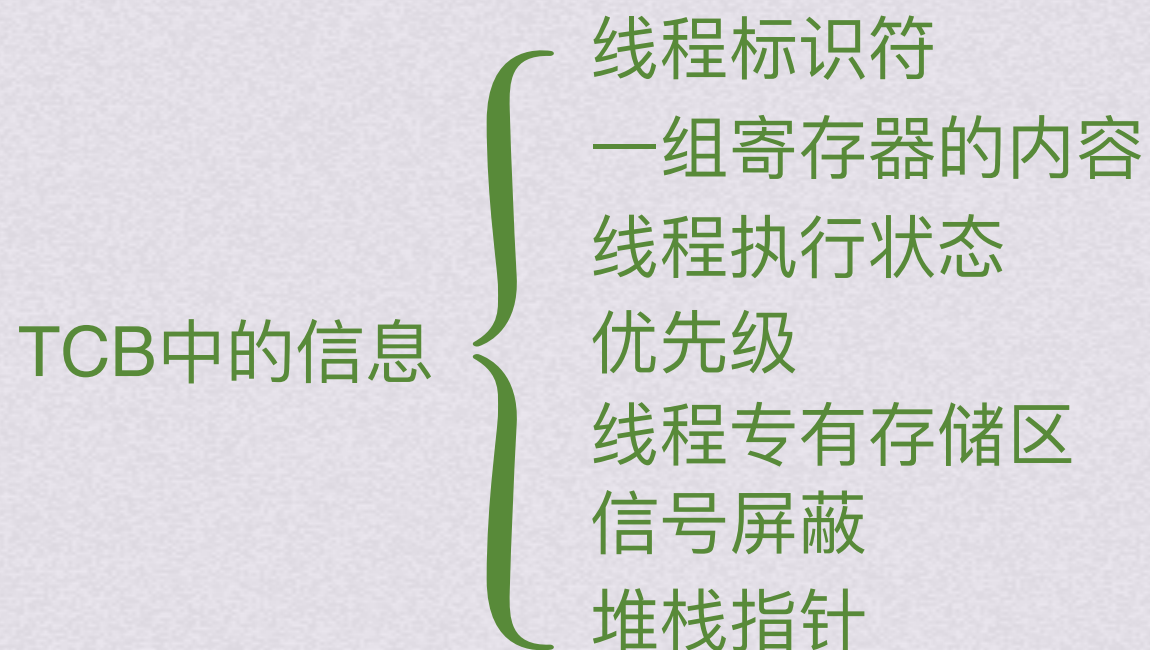


线程管理中的数据结构



线程管理中的数据结构

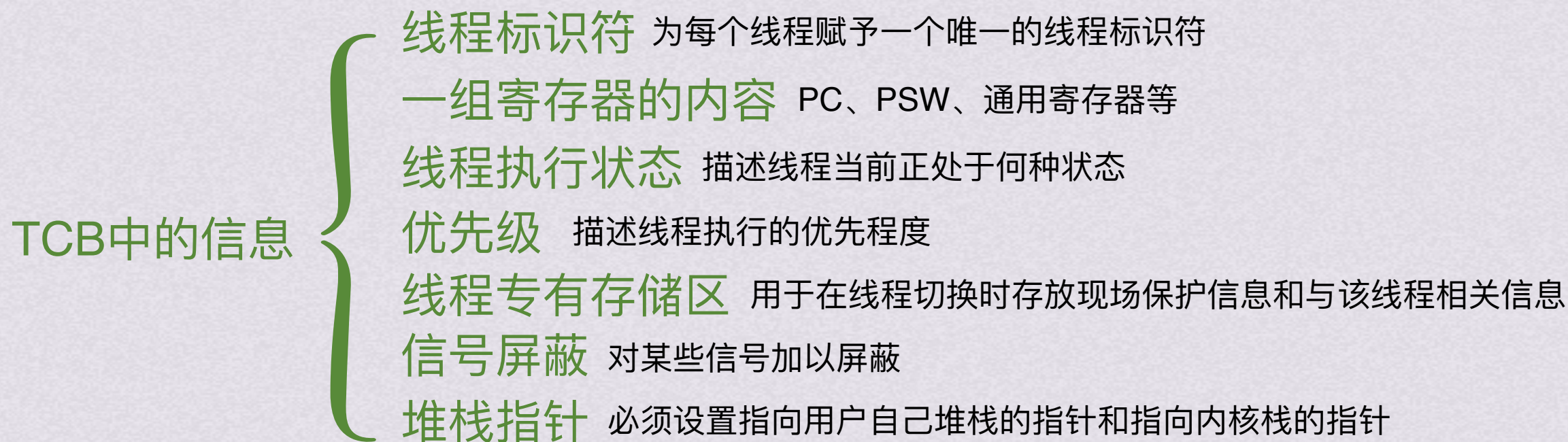
为了便于系统描述和管理进程，OS为每个线程定义了一个数据结构**TCB(线程控制块)**





线程管理中的数据结构

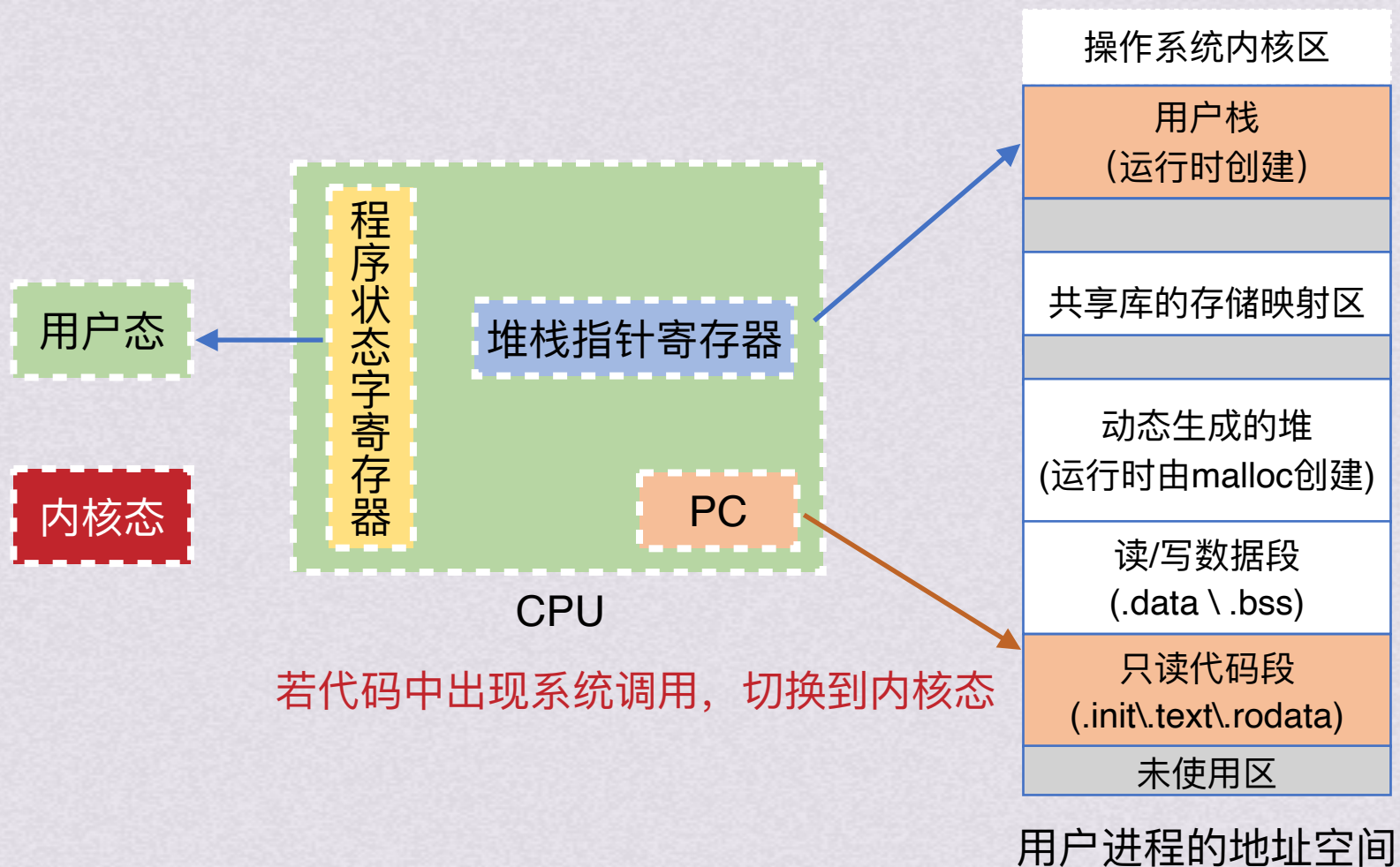
为了便于系统描述和管理线程，OS为每个线程定义了一个数据结构**TCB(线程控制块)**





线程管理中的数据结构

堆栈指针

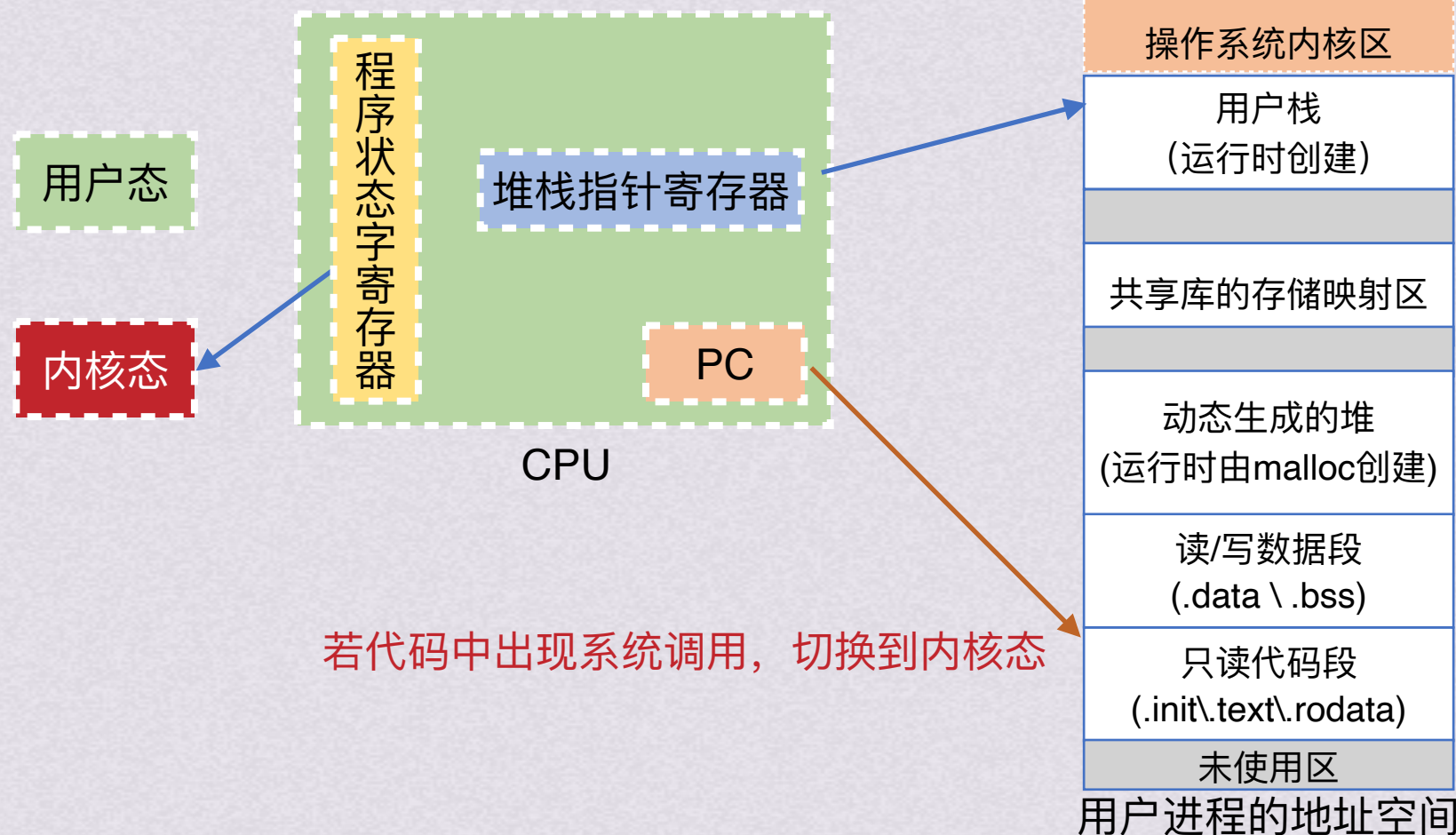




线程管理中的数据结构

堆栈指针

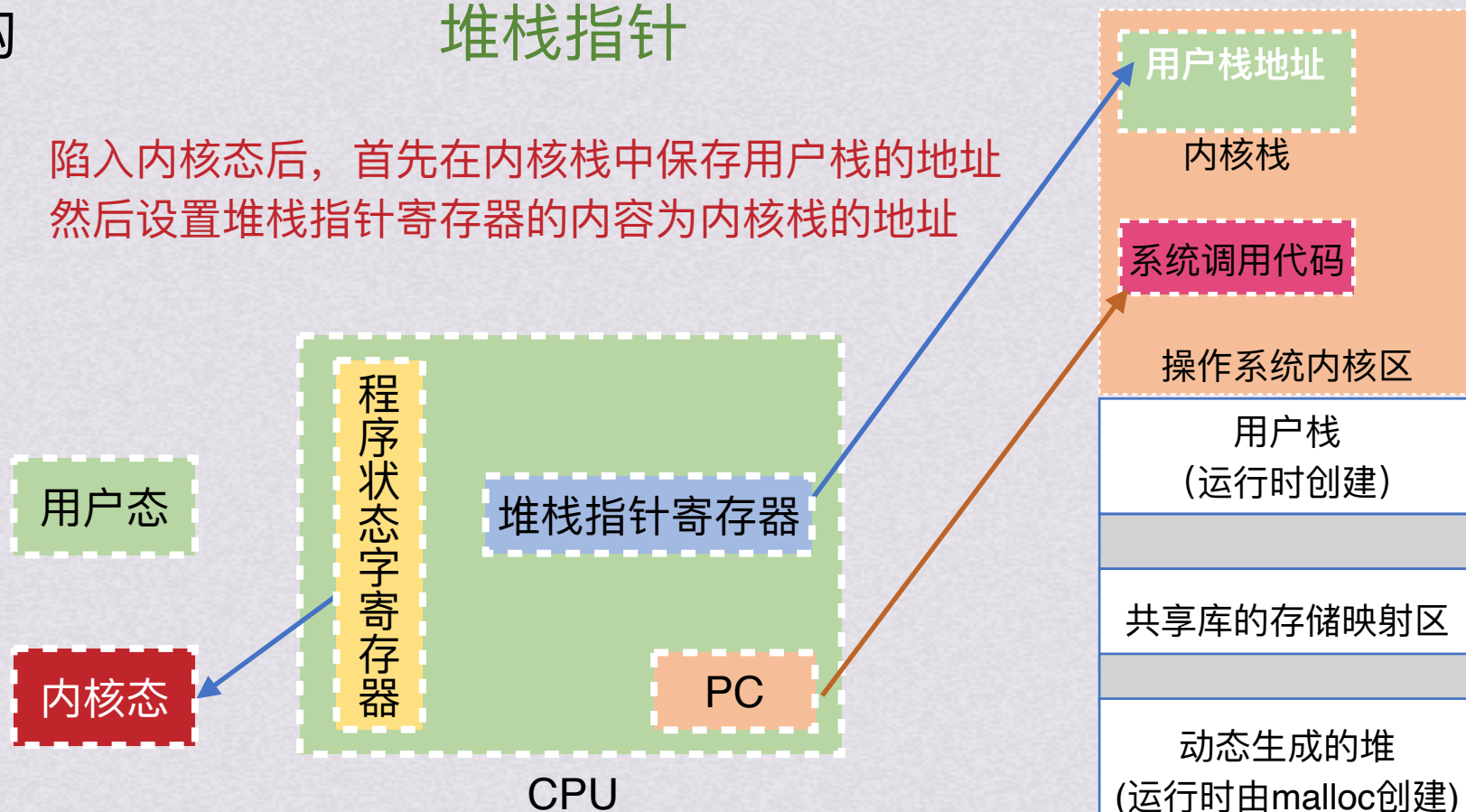
陷入内核态后，首先在内核栈中保存用户栈的地址
然后设置堆栈指针寄存器的内容为内核栈的地址





线程管理中的数据结构

堆栈指针



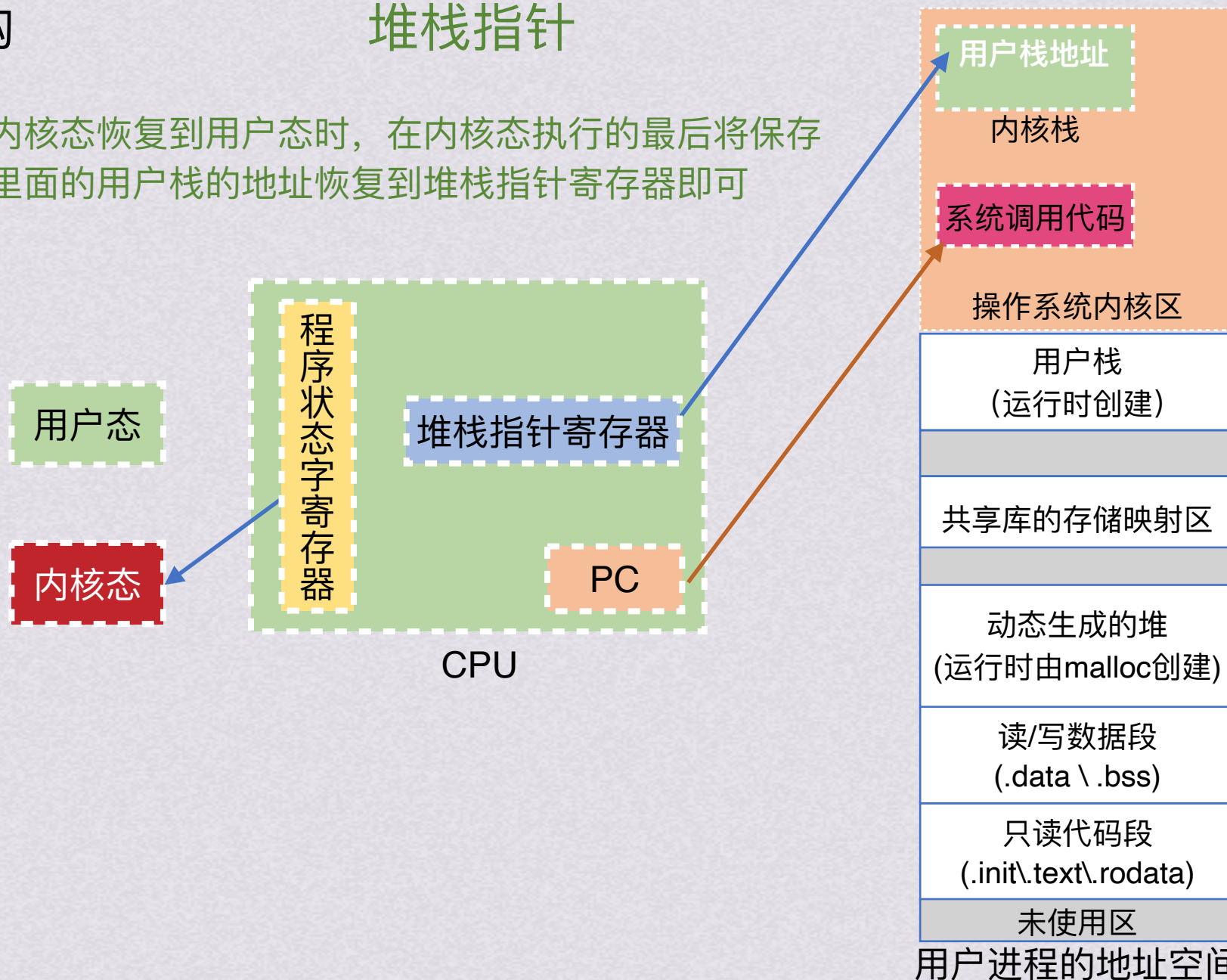
若代码中出现系统调用，切换到内核态



线程管理中的数据结构

堆栈指针

当进程从内核态恢复到用户态时，在内核态执行的最后将保存在内核栈里面的用户栈的地址恢复到堆栈指针寄存器即可

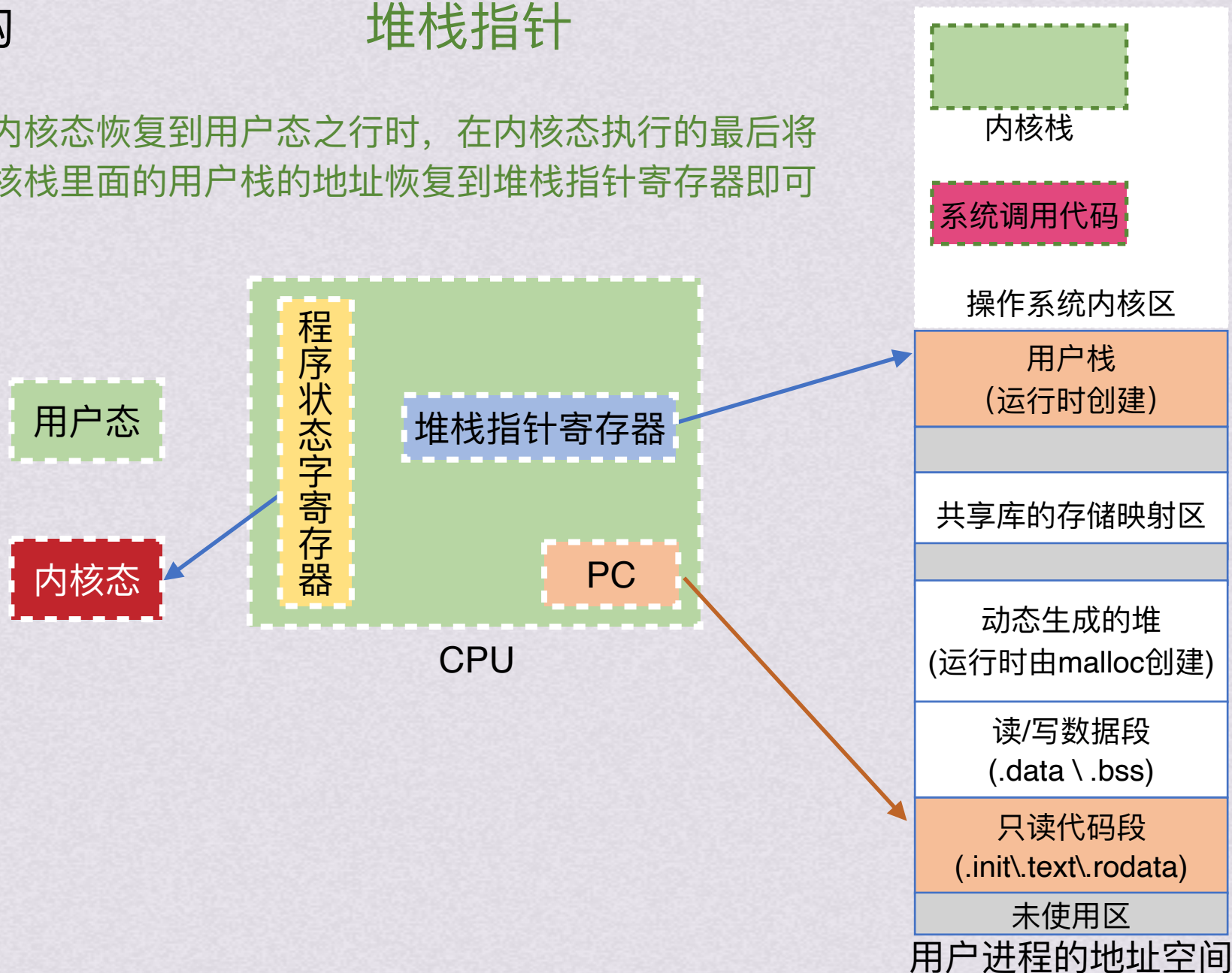




线程管理中的数据结构

堆栈指针

当进程从内核态恢复到用户态之行时，在内核态执行的最后将保存在内核栈里面的用户栈的地址恢复到堆栈指针寄存器即可



线程管理中的数据结构

堆栈指针

区分内核栈和用户栈的原因：每个进程运行时都持有上下文，用于保证并行性。
为了保证内核和用户态隔离，陷入内核不影响用户态，所以使用了不同的栈





线程管理中的数据结构



线程的控制





线程的控制

线程的创建

用户程序在启动时，通常只有一个初始化线程在执行，它利用一个线程创建函数来创建新线程，并返回一个线程标识符

线程的终止

终止线程通过调用相应的函数执行终止操作，大多数OS中，线程被终止后不会立即释放它所占有的所有资源，只有当进程中的其他线程执行了分离函数后，被终止的线程才会与资源分离，此时的资源才能被其他线程利用

已被终止但尚未释放资源的线程，仍可被需要它的线程所调用以恢复运行



线程的控制

线程的创建

用户程序在启动时，通常只有一个初始化线程在执行，它利用一个线程创建函数来创建新线程，并返回一个线程标识符

线程的终止

终止线程通过调用相应的函数执行终止操作，大多数OS中，线程被终止后不会立即释放它所占有的所有资源，只有当进程中的其他线程执行了**分离函数**后，被终止的线程才会与资源分离，此时的资源才能被其他线程利用

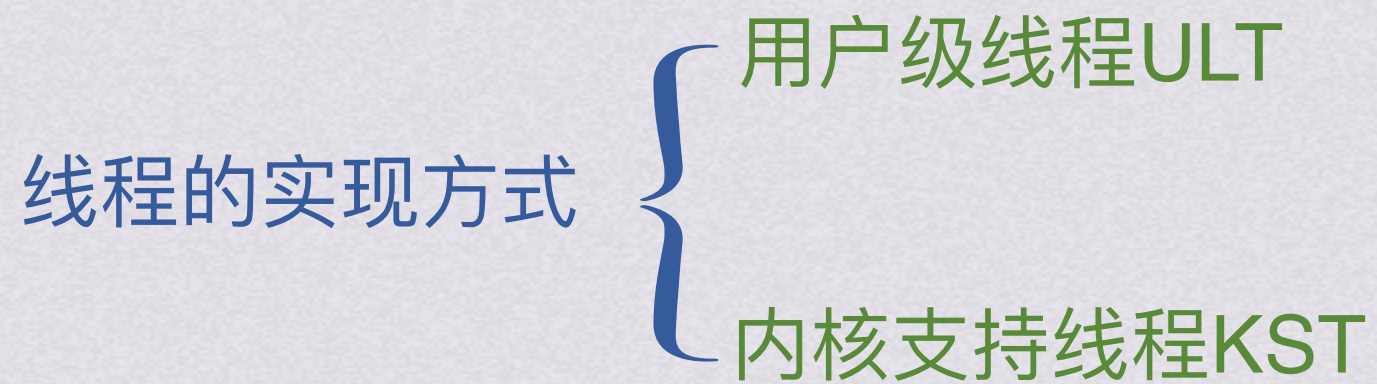
已被终止但尚未释放资源的线程，仍可被需要它的线程所调用以恢复运行



线程的控制



线程的实现方式





线程的实现方式

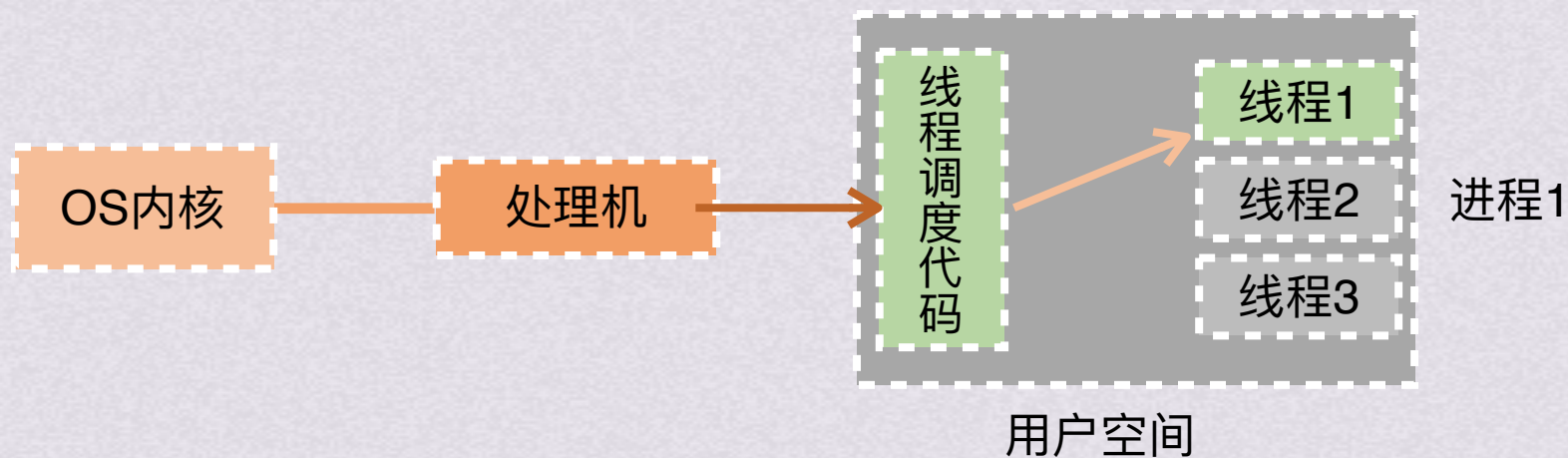
用户级线程ULT



线程的实现方式

用户级线程ULT

把整个线程实现部分放在用户空间中，内核看到的就是一个单线程进程



内核支持线程KST

线程的基本概念

线程的状态与转换

线程管理中的数据结构

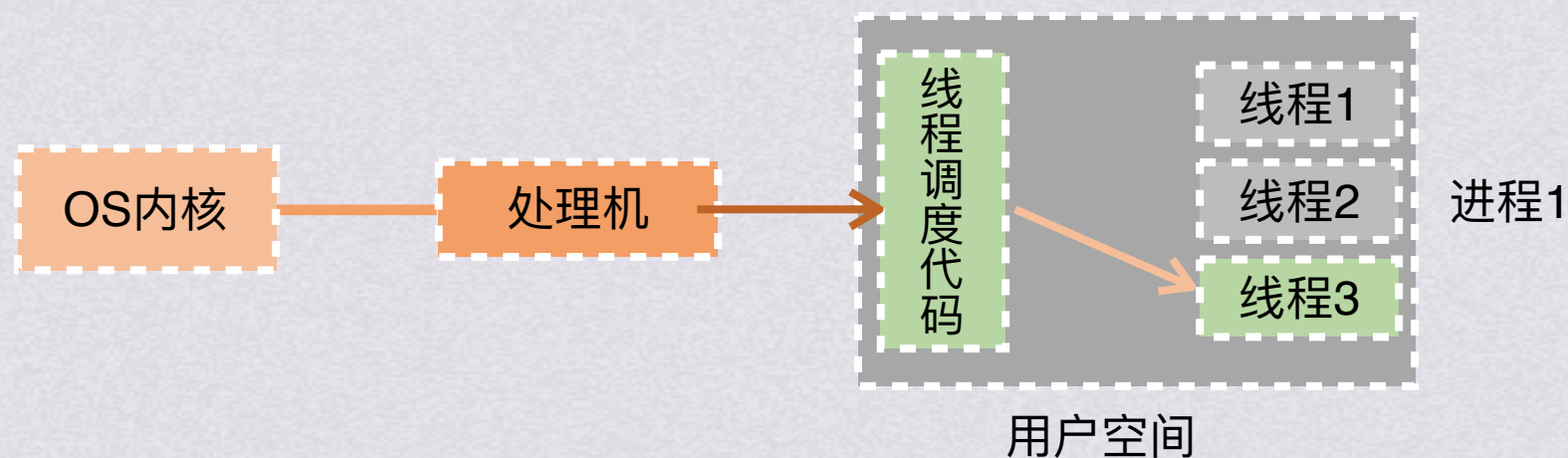
线程的控制



线程的实现方式

用户级线程ULT

把整个线程实现部分放在用户空间中，内核看到的就是一个单线程进程



内核支持线程KST

线程的基本概念

线程的状态与转换

线程管理中的数据结构

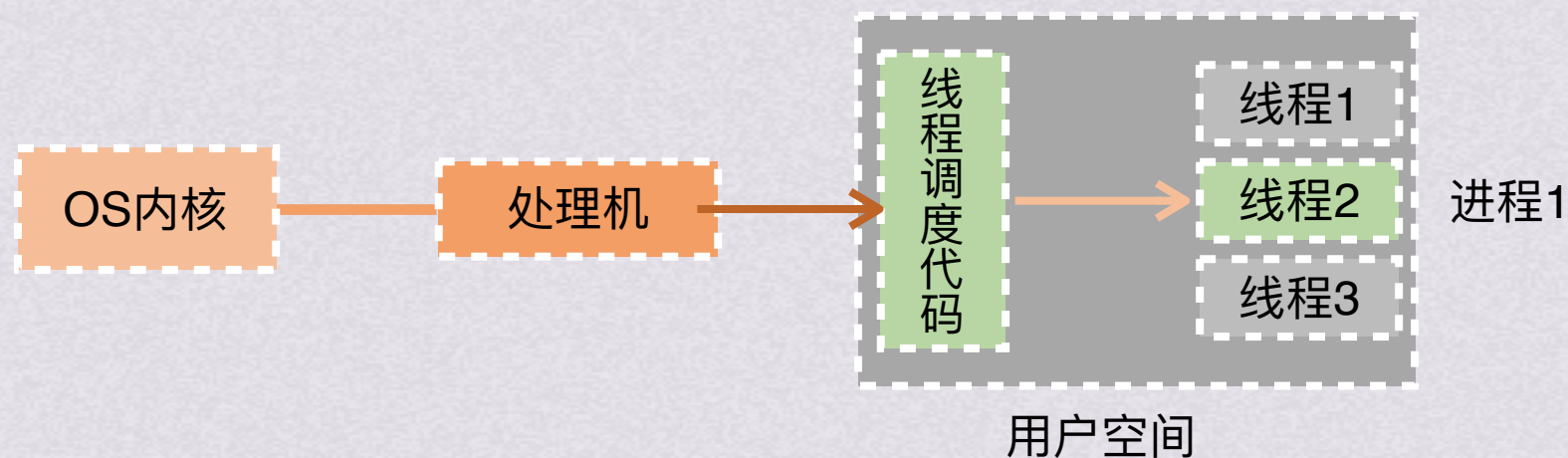
线程的控制



线程的实现方式

用户级线程ULT

把整个线程实现部分放在用户空间中，内核看到的就是一个单线程进程

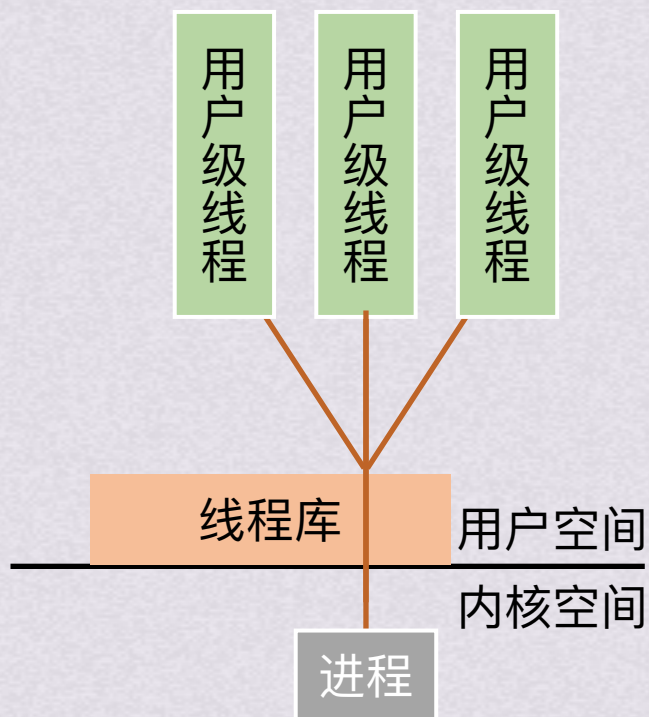




线程的实现方式

用户级线程ULT

把整个线程实现部分放在用户空间中，内核看到的就是一个单线程进程



- 1.线程切换在用户空间实现，无需模式切换
- 2.每个进程可以选择自己专用的调度算法，对自己内部的线程进行调度和管理
- 3.ULT的实现与OS无关，因为ULT管理的代码会显式地出现在用户程序中，因此ULT甚至可以在不支持线程机制的OS平台上实现

- 1.内核只识别到一个进程，因此调度的对象是整个用户进程，每次只能给整个进程分配CPU，进程中只有一个线程能执行，无法发挥多处理器系统优势
- 2.进程中的某个线程发起IO系统调用，会导致进程中的所有线程全部被阻塞

内核支持线程KST

线程的基本概念

线程的状态与转换

线程管理中的数据结构

线程的控制



线程的实现方式

用户级线程ULT

内核支持线程KST

线程的基本概念

线程的状态与转换

线程管理中的数据结构

线程的控制



线程的实现方式

内核支持线程KST

用户级线程ULT

线程的基本概念

线程的状态与转换

线程管理中的数据结构

线程的控制



线程的实现方式 内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知到的线程



用户级线程ULT

线程的基本概念

线程的状态与转换

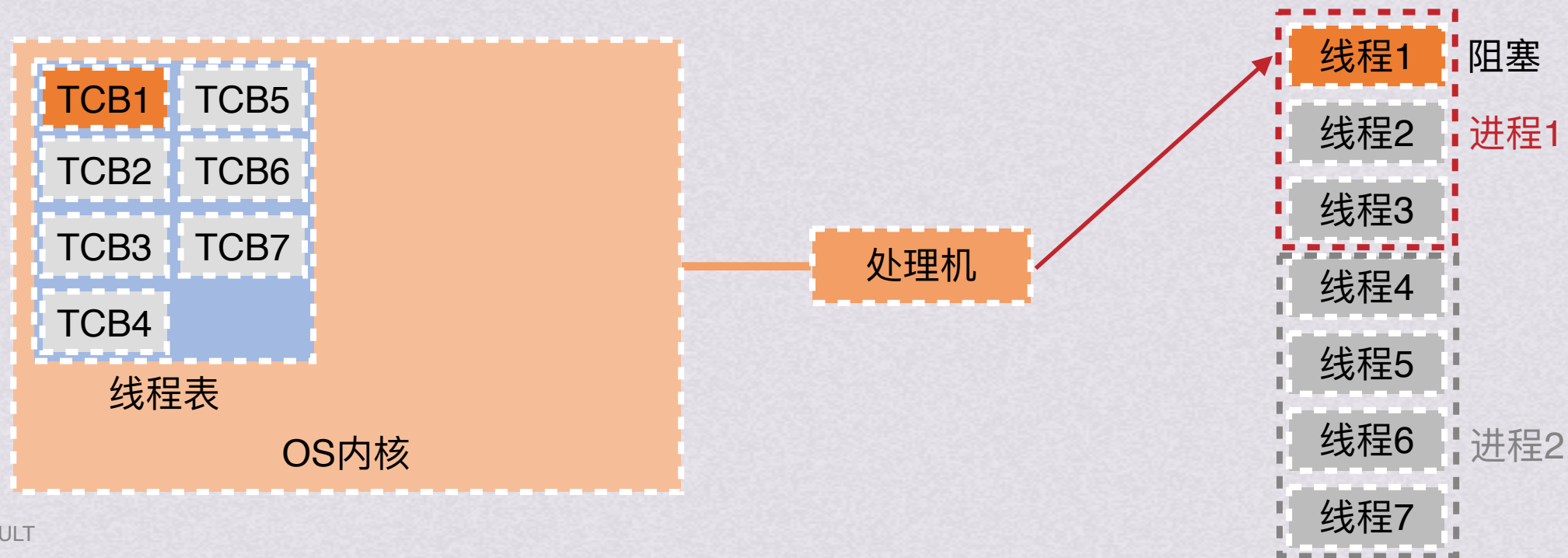
线程管理中的数据结构

线程的控制



线程的实现方式 内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知到的线程



用户级线程ULT

线程的基本概念

线程的状态与转换

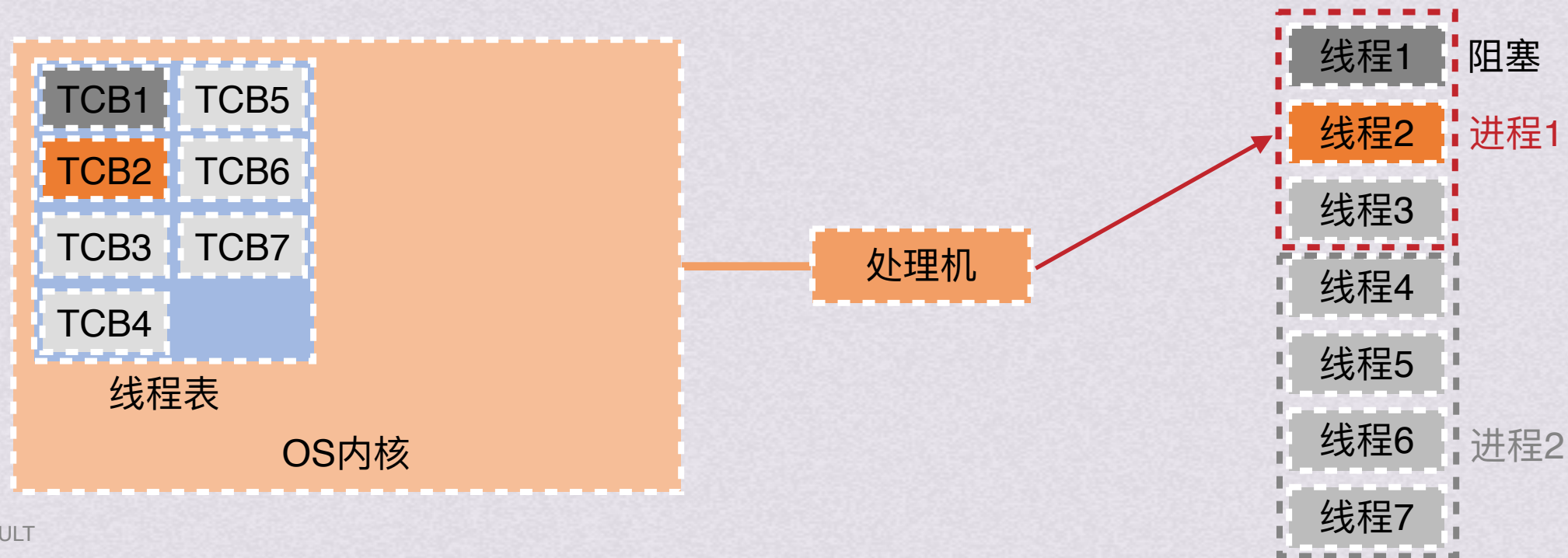
线程管理中的数据结构

线程的控制



线程的实现方式 内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知到的线程



用户级线程ULT

线程的基本概念

线程的状态与转换

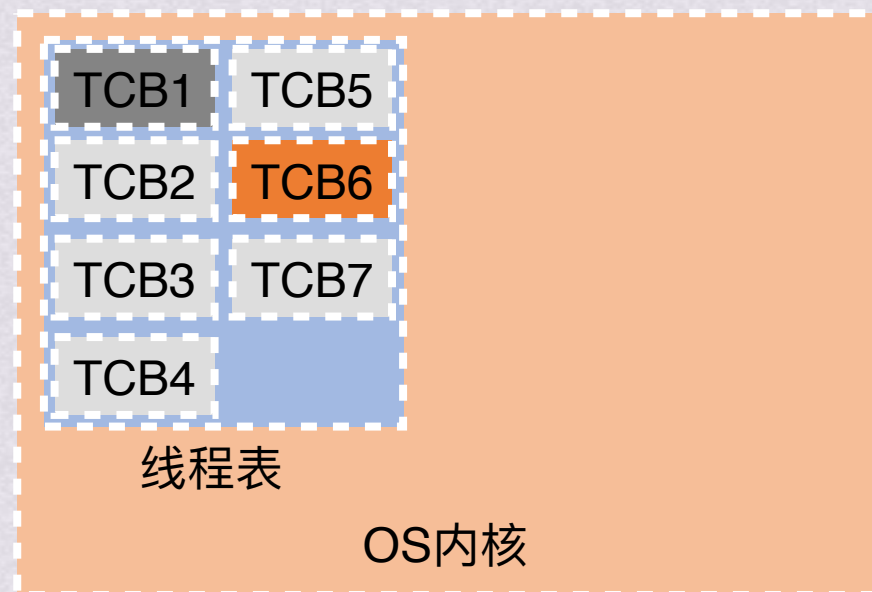
线程管理中的数据结构

线程的控制



线程的实现方式 内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知到的线程



处理机



用户级线程ULT

线程的基本概念

线程的状态与转换

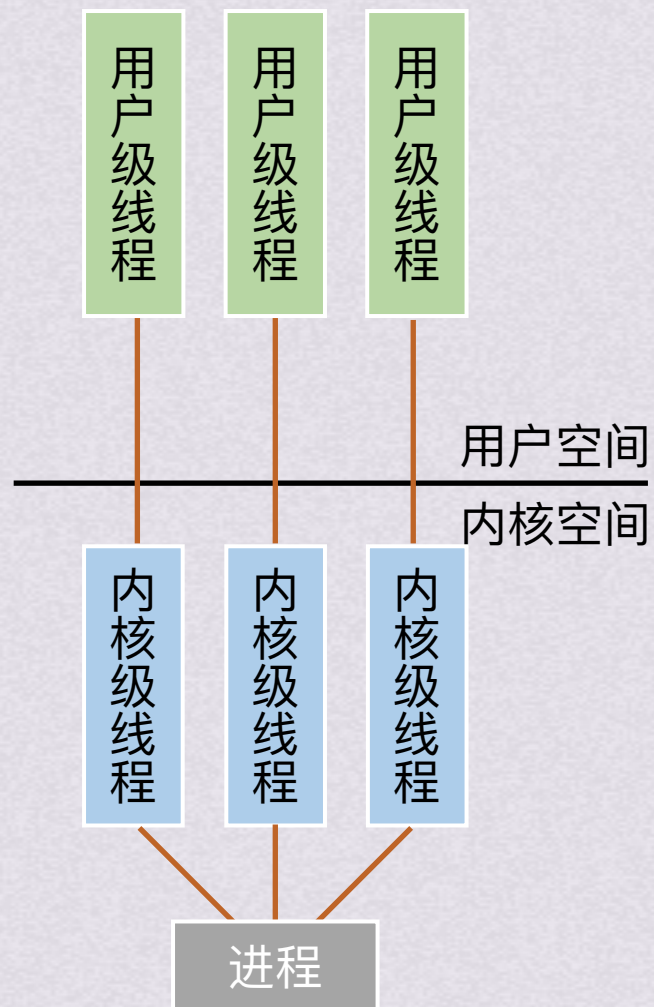
线程管理中的数据结构

线程的控制



线程的实现方式 内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知到的线程



- 1.内核可识别到内核级线程，因此调度的对象是内核级线程，在多处理机系统中，每次可以给不同的内核级线程分配CPU，提高并行度
- 2.如果用户进程中的一个线程被阻塞，内核可以调度该进程内的其他线程上处理机，或者调度其他进程的线程

- 1.每次线程切换都需要CPU变态，开销较大
- 2.TCB存放在内核空间中，内存占用较大

线程的实现方式

	用户级线程ULT	内核支持线程KST
调度	开发者自己实现 内核无法感知	内核来实现 内核可以感知
开销	线程管理开销远小于KST	开销较大

线程只能在进程内部进行资源竞争

线程可以在全系统内进行资源的竞争

- 线程的基本概念
- 线程的状态与转换
- 线程管理中的数据结构
- 线程的控制



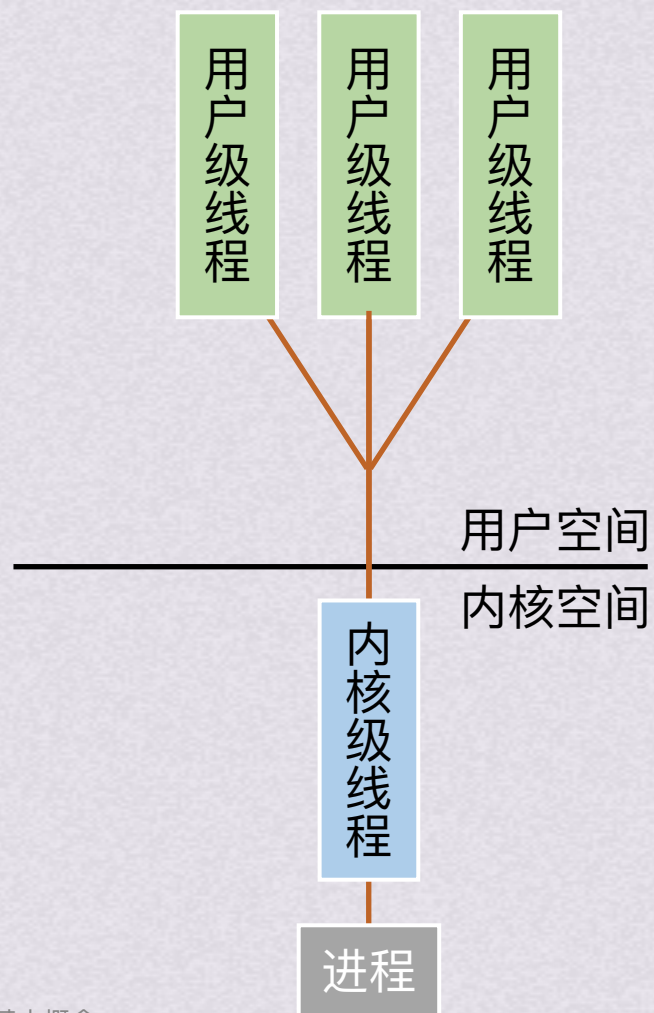
线程的实现方式

同时支持ULT和KST的OS
如何实现线程？

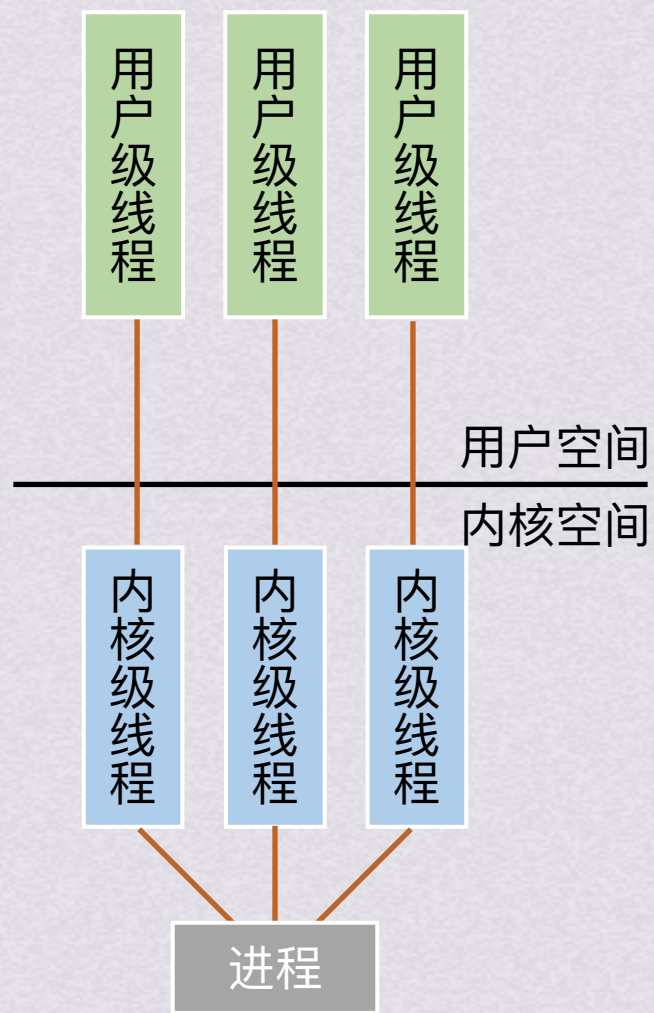


线程的实现方式

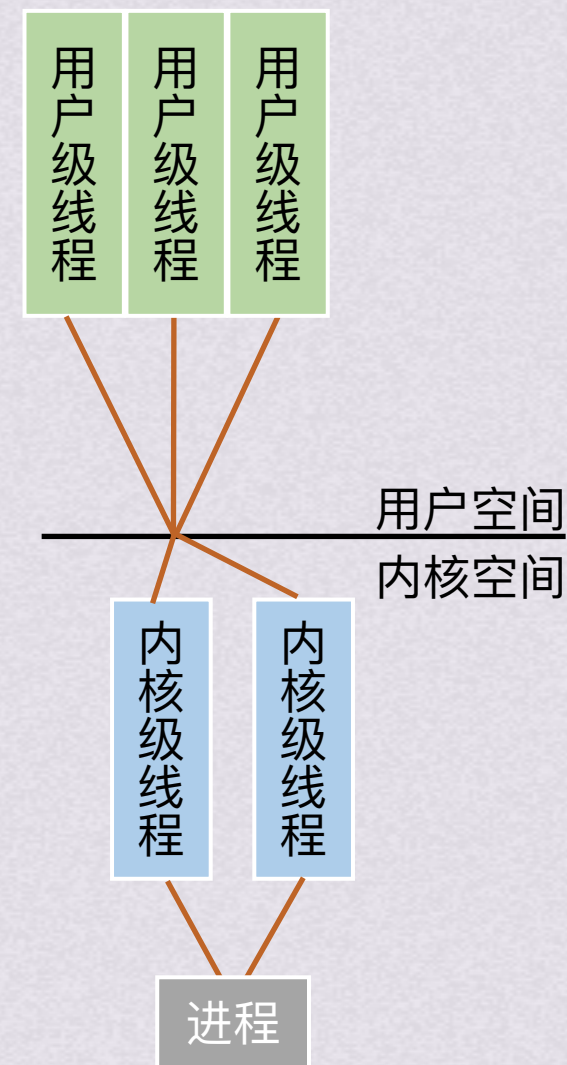
多线程模型



多对一模型



一对一模型



多对多模型



线程的实现方式

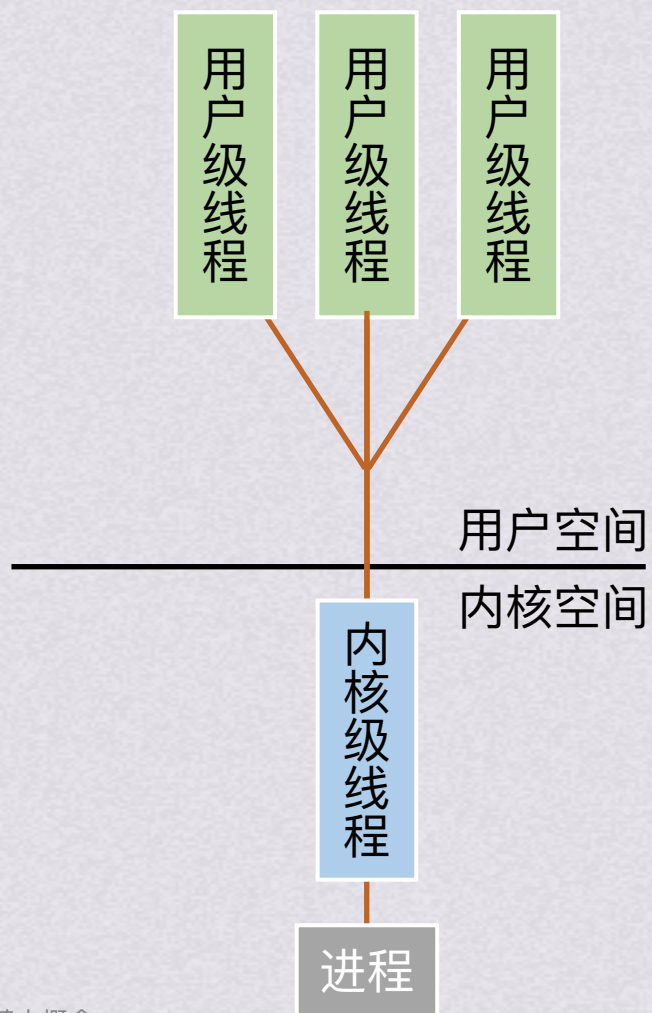
多线程模型

多对一模型

把多个用户级线程映射到一个内核级线程，线程管理在用户空间完成，用户级线程对操作系统不可见

优点：线程管理在用户空间进行，效率高

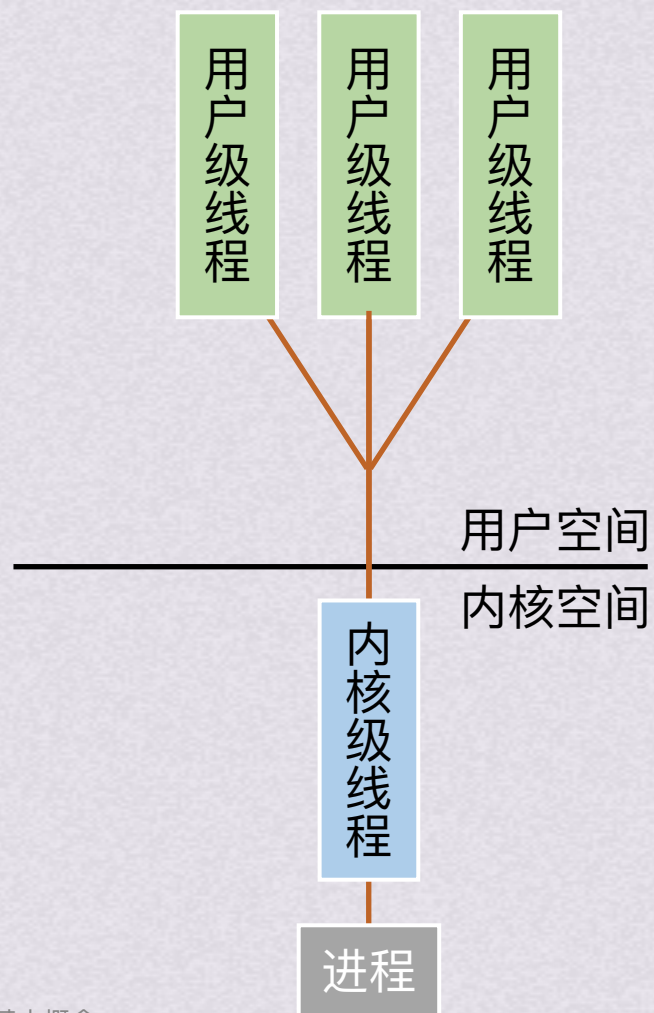
缺点：一个线程在使用内核服务时被阻塞，整个进程都会被阻塞



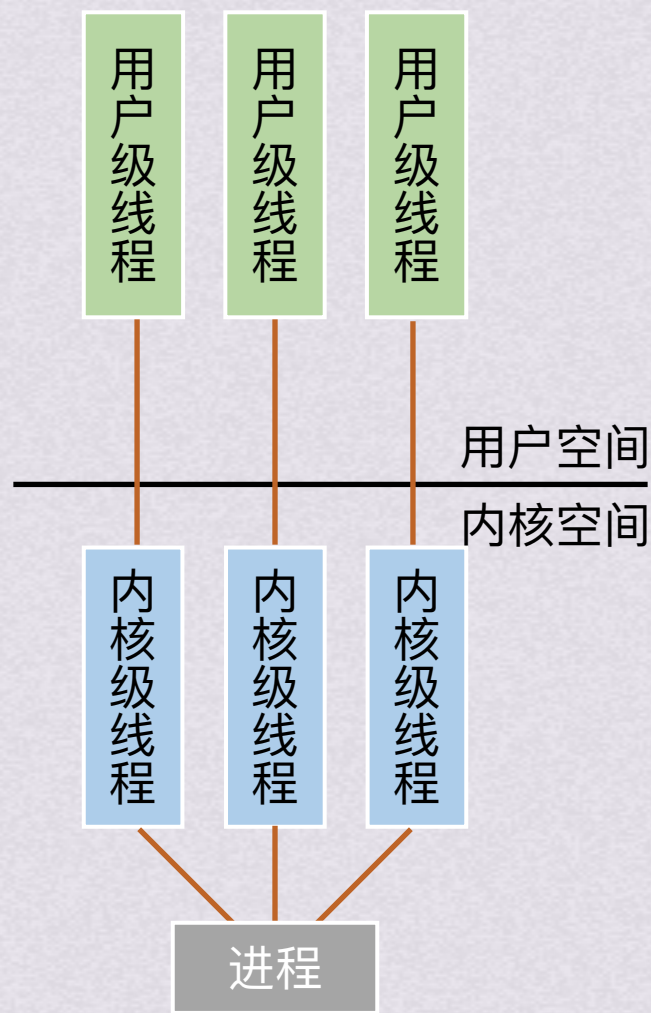


线程的实现方式

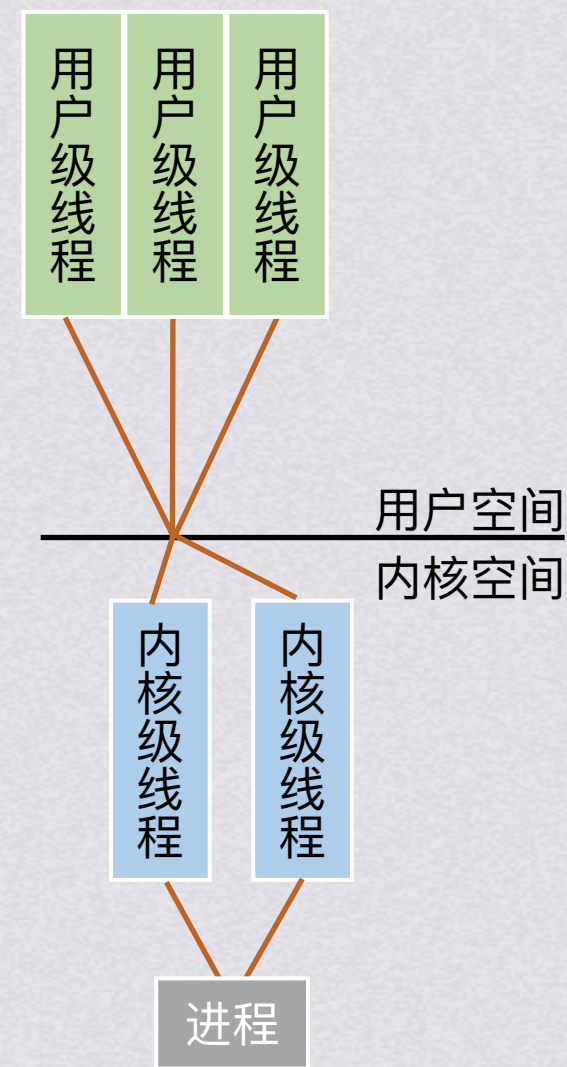
多线程模型



多对一模型



一对一模型



多对多模型



线程的实现方式

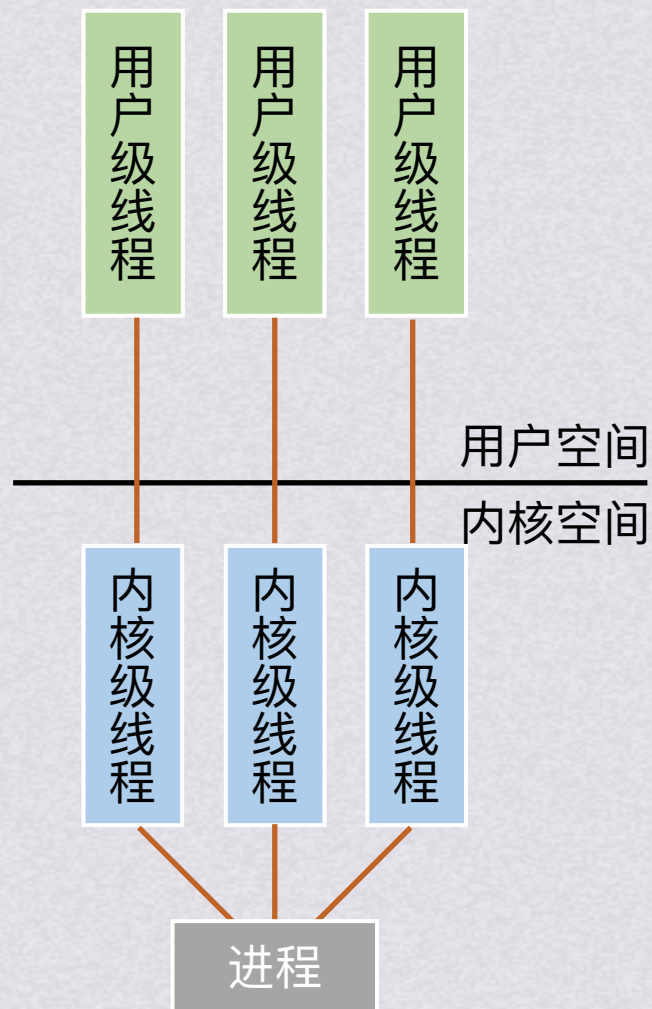
多线程模型

一对一模型

每个用户级线程都映射到一个内核级线程

优点：当一个线程被阻塞时，允许另一个线程继续运行，并发能力强

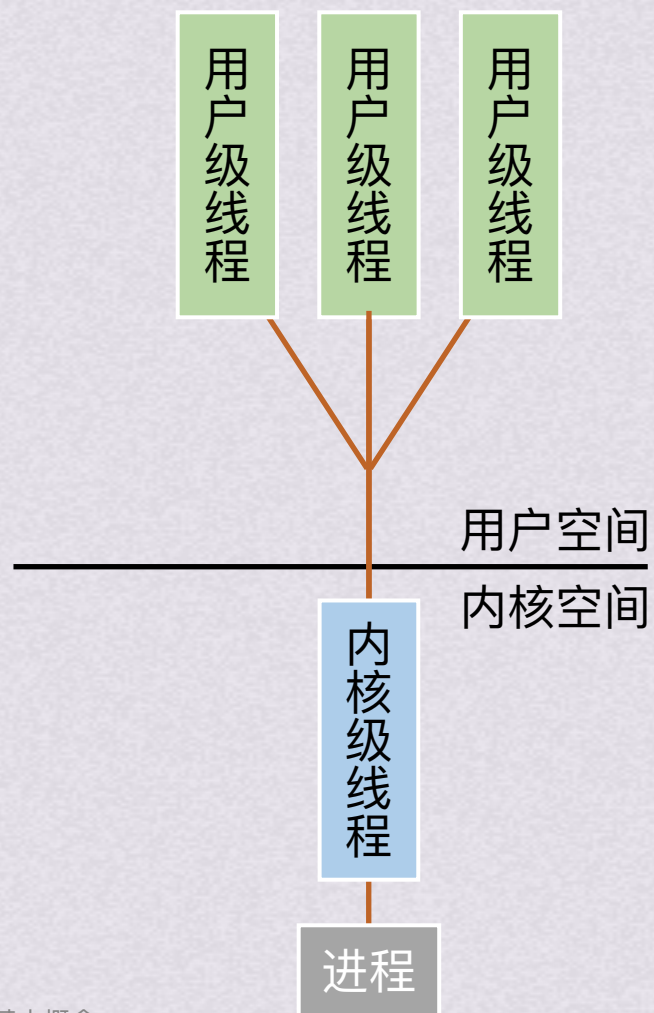
缺点：每个用户级线程都需要一个内核级线程与其对应，线程管理开销很大



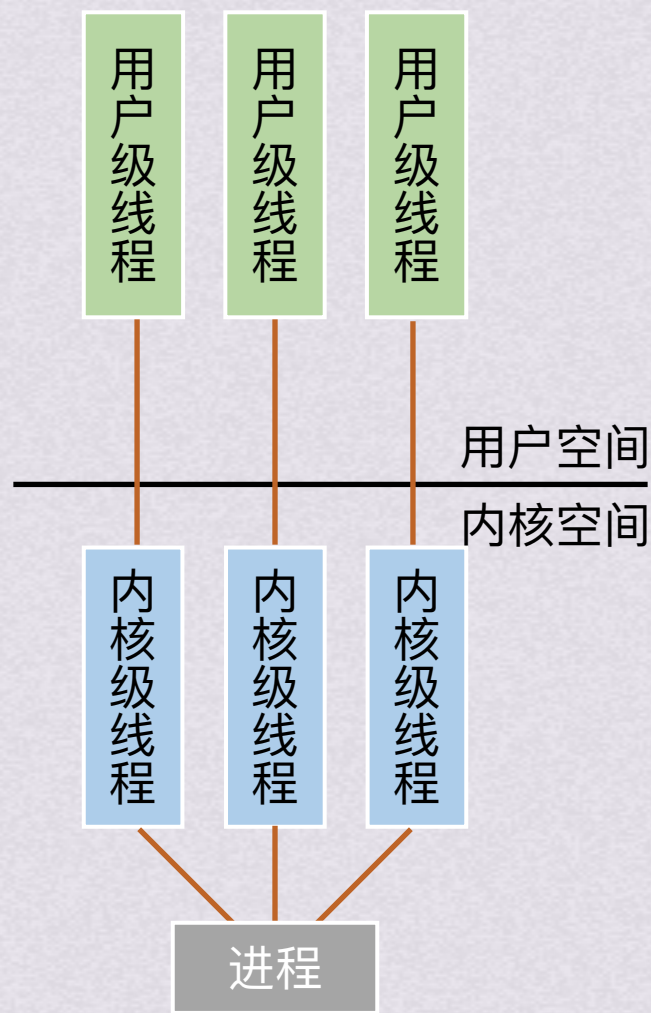


线程的实现方式

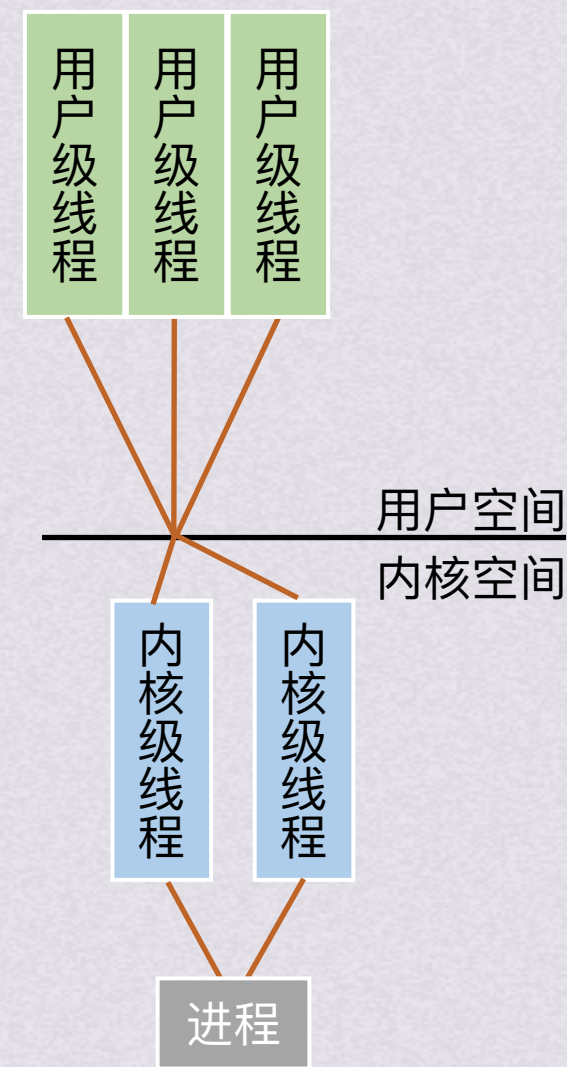
多线程模型



多对一模型



一对一模型

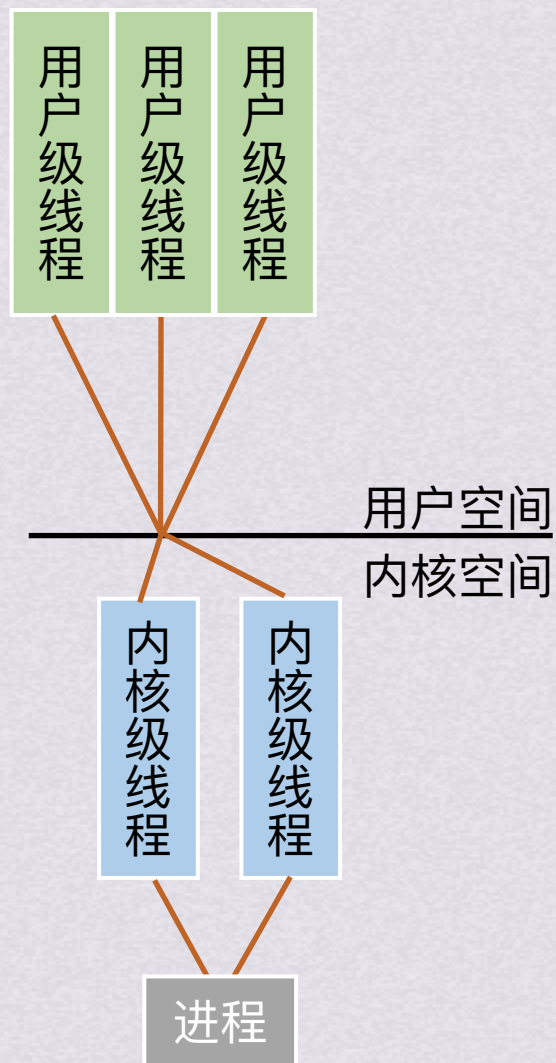


多对多模型



线程的实现方式

多线程模型



多对多模型

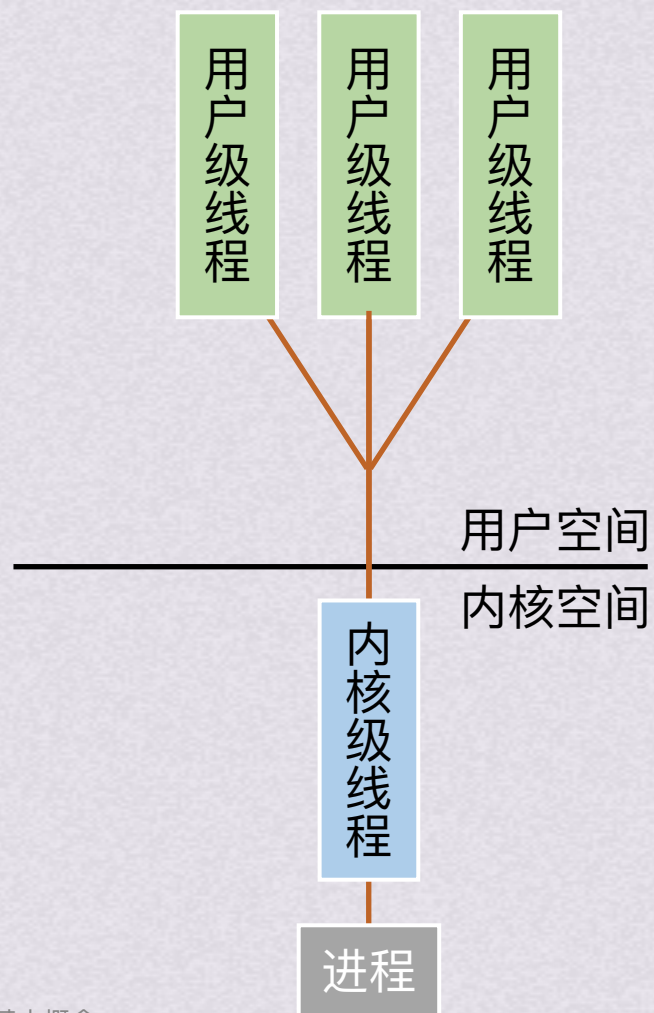
n 个用户级线程映射到 m 个内核级线程上, $m \leq n$

结合一对一模型和多对一模型的灵活使用

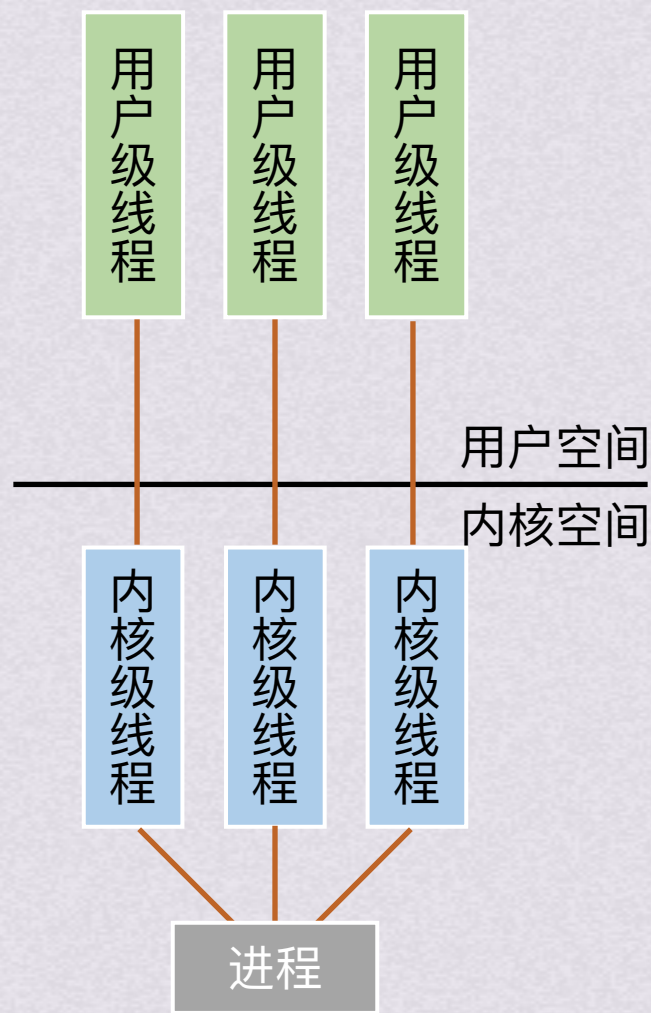


线程的实现方式

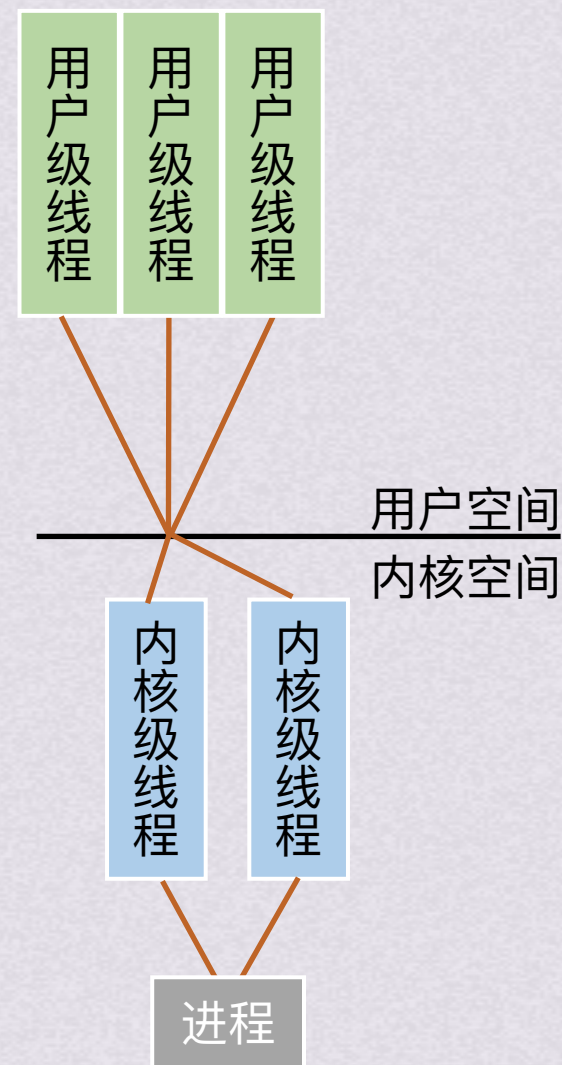
多线程模型



多对一模型



一对一模型



多对多模型